



Proposal: SwayLayoutManager

Laboratorio de Investigación y Desarrollo de
Software Libre (LIDSoL)

Areas:

Frontend

Backend

Date: September 23, 2025

Project Title

SwayLayoutManager: Workspace Layout Manager

Detailed Description

Problem

Users of **-Sway Window Manager-** often organize their applications into specific layouts across multiple workspaces to optimize their workflow. Reopening every app manually and arranging them can be time consuming and tedious.

Proposed Solution

The idea behind **SwayLayoutManager** is to create a versatile application that automates this process through two distinct interfaces: a **command-line interface (CLI)**, perfect for scripting and automation, and an intuitive **graphical user interface (GUI)** for easy visual management. Through either interface, users can save the current window layout (applications, arrangement, and workspace) as a named "template" or "preset." Subsequently, they can load this template via **CLI** with a sway short-cut or through the **GUI** app to instantly restore all applications to their designated positions and workspaces.

Use Case

A developer might have a workspace with a *Visual Studio Code* window occupying the left half of the screen and two `foot` terminals stacked vertically on the right half. With **SwayLayoutManager**, they could save this layout as `dev-workspace`. When starting their computer, they could run a simple command to load this template and have their development environment ready instantly, without needing to open and position each window manually.

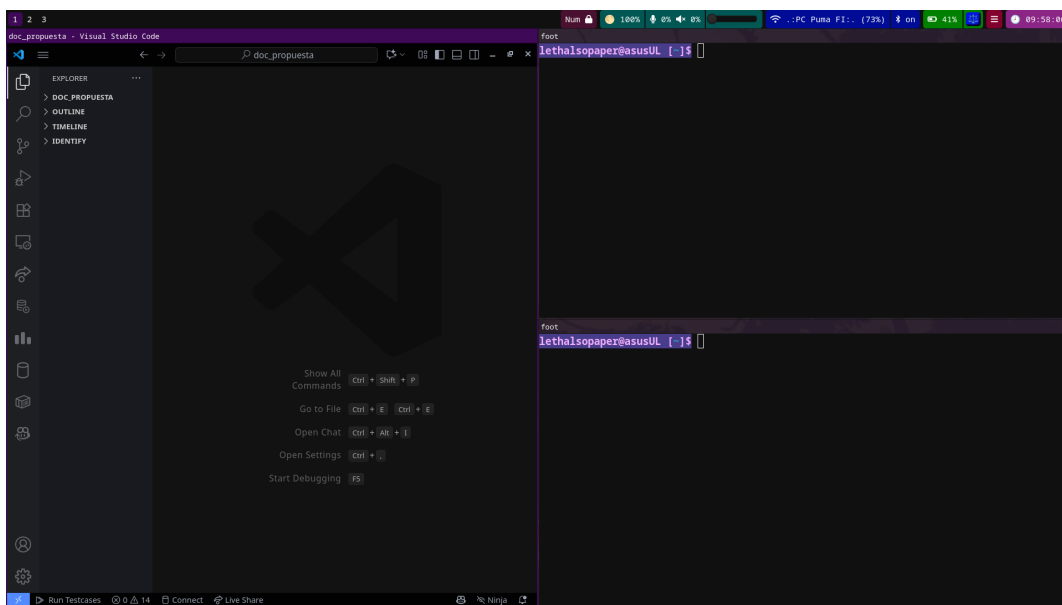


Figure 1: Use case

Requirements

- **Operating System:** Any GNU/Linux distribution with **Wayland** support.
- **Window Manager:** **sway** (version 1.10 or higher).
- **Core CLI (Backend):**
 - Developed in **Go**, compiling to a single executable.
 - Has **no runtime dependencies** for the end-user.
- **GUI (Frontend):**
 - Developed in **Python 3**.
 - Requires a Python 3 interpreter and a GUI toolkit library (e.g., PyQt6, Tkinter) to run.
- **User Knowledge:** Familiarity with the Linux terminal and basic **sway** configuration concepts.

Implementation Details

Internal Operation

The tool will interact directly with the **sway** process via **swaymsg** utility which communicates to the IPC (*Inter-Process Communication*) socket, which allows querying the window manager's state and sending it commands programmatically.

Saving Templates (save)

1. The command **swaymsg -t get_tree** will be executed to get a full JSON dump of **sway**'s current layout tree.
2. The application will parse this JSON to extract relevant information for each window in the specified workspaces: the application identifier (**app_id** or **window_class**), the workspace it belongs to, its layout (horizontal/vertical), and its relative position in the container.
3. This structured information will be saved to a readable configuration file, for example, `~/.config/swaylayoutmanager/presets/template_name.json`.

Loading Templates (load)

1. The tool will read the configuration file for the template specified by the user.
2. It will iterate over each application defined in the template.
3. For each application, it will execute a sequence of **swaymsg** commands to recreate the layout. This includes:
 - Switching to the correct workspace (**swaymsg workspace number <N>**).
 - Launching the application (**swaymsg exec <command>**) if it is not already running.

- Using `sway`'s selection criteria (e.g., `[app_id="..."]`) to focus the newly opened window and move or adjust it to the saved layout.

User Interface (CLI)

The main interaction will be through the terminal, with this subcommand structure:

```
swaylayoutmgr save <name>
```

Captures the current state of all active workspaces and saves it as a named layout preset. The command executes `swaymsg -t get_tree | jq` to extract workspace layouts, window positions, application identifiers and some other relevant information.

This data is saved into a JSON file which will be located in `~/.config/swaylayoutmanager/presets/<name>.json` with metadata for future restoration.

```
swaylayoutmgr load <name>
```

Reads a saved preset and recreates the workspace layout by launching applications and positioning them according to the saved configuration. The process involves:

1. Parsing the preset JSON and validating its structure
2. Switching to each workspace and optionally clearing existing windows
3. Launching missing applications using `swaymsg exec <command>`
4. Applying layout reconstruction through `swaymsg` commands for positioning, resizing, and focus restoration
5. Handling errors gracefully by skipping unavailable applications and logging issues

```
swaylayoutmgr list
```

Displays all available presets with or without metadata including creation date, workspace count, and application summary. The command scans `~/.config/swaylayoutmanager/presets/` for JSON files and validates their integrity and compatibility.

Example Output:

```

1 Available Layout Presets:
2   dev-workspace      (3 workspaces, 5 apps)    [Valid]
3     Created: 2024-12-15 14:30
4
5   media-setup        (2 workspaces, 3 apps)    [Valid]
6     Created: 2024-12-10 09:15
7
8   presentation-mode  (1 workspace, 2 apps)    [Invalid - missing app]
9     Created: 2024-12-01 16:45
```

Listing 1: Layout preset list output

`swaylayoutmgr delete <name>`

Removes a saved preset after user confirmation. Includes `-force` flag for scripting and `-backup` flag for safety.